

# The Petit-Ami Remote Display Protocol

S. A. Franco

August 2026

## Abstract

Remote display mode separates a Petit-Ami program from its display: the program runs on one machine, the client side, and its windows, drawing, widgets and input devices live on another, the server side. This paper defines the wire protocol between the two, carried on the message channels of the network library. Every call of the graphical API has a message; a continuous event stream flows back. The protocol is defined by the header `include/graph_remote.h`, of which this paper is the narrative companion; where the two disagree, the header governs.

## Contents

|           |                                           |          |
|-----------|-------------------------------------------|----------|
| <b>1</b>  | <b>Introduction</b>                       | <b>2</b> |
| <b>2</b>  | <b>Architecture</b>                       | <b>2</b> |
| <b>3</b>  | <b>Transport</b>                          | <b>3</b> |
| <b>4</b>  | <b>Wire format</b>                        | <b>3</b> |
| 4.1       | Menu trees . . . . .                      | 3        |
| 4.2       | Event records . . . . .                   | 4        |
| <b>5</b>  | <b>Session</b>                            | <b>4</b> |
| <b>6</b>  | <b>Calls, queries and ordering</b>        | <b>4</b> |
| <b>7</b>  | <b>Windows</b>                            | <b>4</b> |
| <b>8</b>  | <b>Events and client-local operations</b> | <b>5</b> |
| <b>9</b>  | <b>File transfer</b>                      | <b>5</b> |
| <b>10</b> | <b>Future work</b>                        | <b>5</b> |

## 1 Introduction

The terminal and graphics libraries have always reserved a place for “remote display”: the output of a program encoded and carried over a communications channel, with input events carried back, so that the full functionality of the API survives the trip. The manual states the two conditions that make this possible: all output must be applied in the order issued, and all input events must be received in the order generated, with the rule that outstanding output completes before input is taken.

Two modules implement the mode:

**graph\_client** links with the program. It exposes the complete graphical API; each call composes a message and sends it, and **event()** returns the next inbound event. It needs no threads.

**graph\_server** owns the display. It plugs into the standard graphics package as an overrider, executes inbound call messages against it, and sends the resulting events back out.

The display, the mouse, the keyboard and the joysticks face the user on the server side, while the program logic runs on the client side. Relative to the usual naming, where the machine in front of the user is the client, this arrangement is upside down: the “server” is the screen on the desk, and the “client” may be a machine in another room entirely. The naming here follows the connection roles: the server waits for a connection, the client makes one.

## 2 Architecture

The link is two message channels on adjacent ports, plus a third port reserved for file transfer:

|              |                 |                                                                            |
|--------------|-----------------|----------------------------------------------------------------------------|
| port $p$     | command channel | client $\rightarrow$ server calls, and server $\rightarrow$ client replies |
| port $p + 1$ | event channel   | server $\rightarrow$ client events, continuously                           |
| port $p + 2$ | file port       | stream connection, opened per transfer                                     |

The default command port is 4901.

The server divides its work between two threads. The main thread runs the command loop: wait for a message, execute it against the display library, reply if the call returns values. A second thread runs the event pump: call **event()** forever and send each event out on the event channel. The two directions are asynchronous, which is exactly why they are separate threads; neither ever waits on the other.

The client needs no threads at all. Calls go out on the command channel as they are made, in order. **event()** reads the event channel. A query sends its request and waits for the reply before anything else is issued.

### 3 Transport

The protocol rides the message channels of the network library: `openmsg()` on the client side, `waitmsg()` on the server side, `rdmsg()` and `wrmsg()` to move discrete messages. One protocol message is one channel message; nothing is split across datagrams, and nothing shares one.

Message channels bound the size of a message, found from `maxmsg()` for the address. The write stream chunks itself to fit; every other message must fit the bound with its payload, which in practice bounds string lengths. Senders are responsible for staying under the bound.

The protocol requires a reliable channel in the sense of `relymsg()`: no loss, no duplication, no reordering. The local machine connection of the standard test arrangement is reliable by definition. Operation over an unreliable network needs a retransmission layer beneath this protocol; that layer is future work, and nothing in the message design precludes it.

The secure flag of the message channel calls passes through: a secured deployment secures both channels and the file connections alike.

### 4 Wire format

All multibyte values are little endian. The scalar and composite notations used throughout the catalog:

|                   |                                                     |
|-------------------|-----------------------------------------------------|
| <code>i64</code>  | signed 64 bit integer, the API <code>long</code>    |
| <code>f64</code>  | IEEE 754 double, carrying the API float values      |
| <code>str</code>  | i32 byte length, then the bytes, no terminator      |
| <code>blk</code>  | i32 byte length, then the bytes                     |
| <code>slst</code> | string list: i32 count, then count <code>str</code> |
| <code>menu</code> | menu tree, defined below                            |
| <code>evt</code>  | event record, defined below                         |

Every message begins with a fixed sixteen byte header:

|                  |                  |                                                             |
|------------------|------------------|-------------------------------------------------------------|
| <code>i32</code> | <code>len</code> | total message length in bytes, header included              |
| <code>i16</code> | <code>mid</code> | message id, from the catalog                                |
| <code>i16</code> | <code>seq</code> | request serial, echoed by the reply; 0 where not meaningful |
| <code>i64</code> | <code>wid</code> | window handle the call acts on; 0 where not meaningful      |

The payload follows, holding the call's parameters in the order the API declares them, with the window file parameter excluded: it is the header `wid`.

#### 4.1 Menu trees

A menu serializes depth first: an `i32` count, then that many entries, where each entry is an `i64` flag word (bit 0 the on/off capability, bit 1 the one-of membership, bit 2 the bar), the `i64` id, the `str` face text, and then a nested menu for the branch, empty if the entry has none. An empty tree is a single zero count, and removes the menu where that has meaning.

`stdmenu()` is the one call that returns a menu: the client sends the standard selections and the program's additions, and the server returns the combined tree as it built it. The client materializes that tree as real menu records for the program, so the program can hand it back to `menu()` later, as the API contract requires.

## 4.2 Event records

An event is a fixed record of eleven `i64`: the window handle the event belongs to, the event code, the handled flag, and eight parameter slots holding the fields of the event union in declaration order. A character rides in the first slot. The slots not used by an event code are zero. The fixed layout spends a few bytes to buy a single marshaling routine for every event.

## 5 Session

The first message on the command channel is the hello: the client sends its protocol version and the server replies with its own. A server that does not speak the client's version replies with its version and closes; the client raises the failure to the program. The first message on the event channel is the event-channel hello from the client, which is how the server learns where events go; it carries the version again and has no reply.

The client announces the end of the session with a bye on the command channel; the server drops the connection. A server error that cannot be returned as a reply travels to the client as an error message on the event channel, carrying the error text; the client raises it to the program.

## 6 Calls, queries and ordering

Calls divide in two. A command carries no result: it is sent and the client proceeds. A query returns values: the client sends the request and waits on the command channel for the reply before issuing anything else.

There is therefore at most one request outstanding at any time. Replies cannot interleave with each other or with commands, and the server may process every message in arrival order with no reassembly. The `seq` serial in the header is echoed by the reply purely as a consistency check; a mismatch is a protocol failure.

The ordinary output of a program, the characters written to a window file, travels as the write stream message: a block of bytes exactly as they would have gone to the window, chunked by the sender to the channel bound. Order is preserved end to end: the write stream and the drawing calls arrive in the order the program issued them, which satisfies the manual's first condition for remote display. The event stream preserves generation order, which satisfies the second.

## 7 Windows

Window handles are small integers assigned by the client. Handle 1 is the main window, the standard input and output pair, and exists from the hello. `openwin()` carries the parent handle,

the new handle, and the logical window id; the server maps each handle to a window file of its own. Every subsequent call names its window by handle in the message header. Closing a window file on the client side sends the close message, and the server closes its side.

## 8 Events and client-local operations

The server's event pump thread calls `event()` on the display library without pause, translating each event to the wire record and sending it. The client's `event()` reads the event channel, translates back, and returns the event to the program, replacing the wire handle with the window identification the API promises.

Three operations act on the client's own event stream and put nothing on the wire: `eventover()` and `eventsover()`, which hook event delivery, hook the client's delivery; and `sendevent()`, which injects an event, injects into the client's queue. The input device queries, the number of mice, buttons, joysticks and function keys, are synchronous queries like any others; the devices themselves are the server's, and their activity arrives as events.

## 9 File transfer

Files named in calls live where the program lives, on the client; the picture files of `loadpict()` are the case in point. The server holds a cache, its current directory. When a call names a file, the server looks it up: first the name as given, then the base name in the cache. If it holds the file, the call proceeds.

If not, the server requests the file, on the model of ftp's control and data split. The request goes by message on the command channel, inside the reply window of the pending call, so the single threaded client is guaranteed to be listening. The client answers with a port number, then listens on that port; the server opens the port as a stream connection with `opennet()` and receives the file as raw bytes, close delimited, exactly as an ftp data connection carries one file. The server stores the file in its cache under its base name, never under a path the client supplied, and the pending call then proceeds against the cache and sends its reply.

In this protocol version the port named is always the file port, the command port plus two; the message names it explicitly so the mechanism stays general. One transfer at a time falls out of the one outstanding request rule. The client cannot listen before it sends the port message, so the server retries the connection briefly to cover the gap.

The exchange is deliberately not tied to pictures: any call that names a file uses it, and it stands as the protocol's general bulk transfer mechanism.

## 10 Future work

The protocol as specified assumes a reliable channel and same width, little endian hosts at both ends, which covers the intended deployments and the standard test arrangement. Three extensions are anticipated and none disturb the message catalog: a retransmission layer beneath the protocol for unreliable networks; print files (`openprint()`), reserved in the catalog, whose composition would

run on either side; and big endian hosts, which would swap on the wire boundary. Multiple clients to one server, and compression of the write stream, remain open questions.

## 11 Message catalog

The catalog below is generated from `include/graph_remote.h`, which is the normative definition. Payload notation is as defined in the wire format section; “→” introduces the reply payload where the call is a query.

| Message      | Id | Payload → reply                                                                                                                                                                                                                                       |
|--------------|----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| GR_MHELLO    | 1  | first on the command channel: i64 version → i64 version. The server refuses a version it does not speak by replying its own and closing.                                                                                                              |
| GR_MEVOPEN   | 2  | first on the event channel, from the client, so the server learns where events go: i64 version. No reply.                                                                                                                                             |
| GR_MBYE      | 3  | client is done; the server drops the connection. No payload.                                                                                                                                                                                          |
| GR_MERROR    | 4  | server → client on the event channel: str message. The client raises the error to the program.                                                                                                                                                        |
| GR_MFILEREQ  | 5  | server → client on the command channel, inside the reply window of a pending call that names a file the server does not hold: str name. The client answers with GR_MFILEPORT.                                                                         |
| GR_MFILEPORT | 6  | client → server: i64 port. The client then listens on that port (the file port, command port + 2); the server opens it as a stream connection and receives the file, raw bytes, close delimited, then completes the pending call and sends its reply. |
| GR_MREPLY    | 9  | the reply to any query: the request’s seq, and the payload the request’s catalog entry gives.                                                                                                                                                         |
| GR_MWRITE    | 10 | the write stream of the window: blk of characters, as they would go to the window file. Chunked by the sender to the channel maximum. No reply.                                                                                                       |
| GR_MCLOSEWIN | 11 | the window file closes: the server closes its side. No payload.                                                                                                                                                                                       |
| GR_MCURSOR   | 20 | i64 x, i64 y                                                                                                                                                                                                                                          |
| GR_MMAXX     | 21 | → i64                                                                                                                                                                                                                                                 |
| GR_MMAXY     | 22 | → i64                                                                                                                                                                                                                                                 |
| GR_MHOME     | 23 | (no payload)                                                                                                                                                                                                                                          |
| GR_MDEL      | 24 | (no payload)                                                                                                                                                                                                                                          |
| GR_MUP       | 25 | (no payload)                                                                                                                                                                                                                                          |
| GR_MDOWN     | 26 | (no payload)                                                                                                                                                                                                                                          |
| GR_MLEFT     | 27 | (no payload)                                                                                                                                                                                                                                          |
| GR_MRIGHT    | 28 | (no payload)                                                                                                                                                                                                                                          |

| Message         | Id  | Payload → reply                  |
|-----------------|-----|----------------------------------|
| GR_MBLINK       | 29  | i64 e                            |
| GR_MREVERSE     | 30  | i64 e                            |
| GR_MUNDERLINE   | 31  | i64 e                            |
| GR_MSUPERSCRIPT | 32  | i64 e                            |
| GR_MSUBSCRIPT   | 33  | i64 e                            |
| GR_MITALIC      | 34  | i64 e                            |
| GR_MBOLD        | 35  | i64 e                            |
| GR_MSTRIKEOUT   | 36  | i64 e                            |
| GR_MSTANDOUT    | 37  | i64 e                            |
| GR_MFCOLOR      | 38  | i64 color                        |
| GR_MBCOLOR      | 39  | i64 color                        |
| GR_MAUTO        | 40  | i64 e                            |
| GR_MCURVIS      | 41  | i64 e                            |
| GR_MSCROLL      | 42  | i64 x, i64 y                     |
| GR_MCURX        | 43  | → i64                            |
| GR_MCURY        | 44  | → i64                            |
| GR_MCUREND      | 45  | → i64                            |
| GR_MSELECT      | 46  | i64 u, i64 d                     |
| GR_MTIMER       | 47  | i64 i, i64 t, i64 r              |
| GR_MKILLTIMER   | 48  | i64 i                            |
| GR_MMOUSE       | 49  | → i64                            |
| GR_MMOUSEBUTTON | 50  | i64 m → i64                      |
| GR_MJOYSTICK    | 51  | → i64                            |
| GR_MJOYBUTTON   | 52  | i64 j → i64                      |
| GR_MJOYAXIS     | 53  | i64 j → i64                      |
| GR_MSETTAB      | 54  | i64 t                            |
| GR_MRESTAB      | 55  | i64 t                            |
| GR_MCLRTAB      | 56  | (no payload)                     |
| GR_MFUNKEY      | 57  | → i64                            |
| GR_MFRAMETIMER  | 58  | i64 e                            |
| GR_MAUTOHOLD    | 59  | i64 e; wid 0, the hold is global |
| GR_MWRTSTR      | 60  | str                              |
| GR_MWRTSTRN     | 61  | str                              |
| GR_MSIZBUF      | 62  | i64 x, i64 y                     |
| GR_MTITLE       | 63  | str                              |
| GR_MFCOLORC     | 64  | i64 r, i64 g, i64 b              |
| GR_MBCOLORC     | 65  | i64 r, i64 g, i64 b              |
| GR_MMAXXG       | 100 | → i64                            |
| GR_MMAXYG       | 101 | → i64                            |
| GR_MCURXG       | 102 | → i64                            |
| GR_MCURYG       | 103 | → i64                            |
| GR_MLINE        | 104 | i64 x1, y1, x2, y2               |
| GR_MLINEWIDTH   | 105 | i64 w                            |
| GR_MLINESTYLE   | 106 | i64 style                        |
| GR_MRECT        | 107 | i64 x1, y1, x2, y2               |
| GR_MFRECT       | 108 | i64 x1, y1, x2, y2               |
| GR_MRRECT       | 109 | i64 x1, y1, x2, y2, xs, ys       |

| Message       | Id  | Payload → reply            |
|---------------|-----|----------------------------|
| GR_MFRRECT    | 110 | i64 x1, y1, x2, y2, xs, ys |
| GR_MELLIPSE   | 111 | i64 x1, y1, x2, y2         |
| GR_MFELLIPSE  | 112 | i64 x1, y1, x2, y2         |
| GR_MARC       | 113 | i64 x1, y1, x2, y2, sa, ea |
| GR_MFARC      | 114 | i64 x1, y1, x2, y2, sa, ea |
| GR_MFCHORD    | 115 | i64 x1, y1, x2, y2, sa, ea |
| GR_MFTRIANGLE | 116 | i64 x1, y1, x2, y2, x3, y3 |
| GR_MCURSORG   | 117 | i64 x, i64 y               |
| GR_MBASELINE  | 118 | → i64                      |
| GR_MSETPIXEL  | 119 | i64 x, i64 y               |
| GR_MFOVER     | 120 | (no payload)               |
| GR_MBOVER     | 121 | (no payload)               |
| GR_MFINVIS    | 122 | (no payload)               |
| GR_MBINVIS    | 123 | (no payload)               |
| GR_MFXOR      | 124 | (no payload)               |
| GR_MBXOR      | 125 | (no payload)               |
| GR_MFAND      | 126 | (no payload)               |
| GR_MBAND      | 127 | (no payload)               |
| GR_MFOR       | 128 | (no payload)               |
| GR_MBOR       | 129 | (no payload)               |
| GR_MCHRSIZX   | 130 | → i64                      |
| GR_MCHRSIZY   | 131 | → i64                      |
| GR_MFONTS     | 132 | → i64                      |
| GR_MFONT      | 133 | i64 fc                     |
| GR_MFONTNAM   | 134 | i64 fc → str               |
| GR_MFONTSIZ   | 135 | i64 s                      |
| GR_MSETPOINTS | 136 | f64 ps                     |
| GR_MPOINTS    | 137 | → f64                      |
| GR_MCHRSPCY   | 138 | i64 s                      |
| GR_MCHRSPCX   | 139 | i64 s                      |
| GR_MDPMX      | 140 | → i64                      |
| GR_MDPMY      | 141 | → i64                      |
| GR_MSTRSIZ    | 142 | str → i64                  |
| GR_MCHRPOS    | 143 | str, i64 p → i64           |
| GR_MWRITEJUST | 144 | str, i64 n                 |
| GR_MJUSTPOS   | 145 | str, i64 p, i64 n → i64    |
| GR_MCONDENSED | 146 | i64 e                      |
| GR_MEXTENDED  | 147 | i64 e                      |
| GR_MXLIGHT    | 148 | i64 e                      |
| GR_MLIGHT     | 149 | i64 e                      |
| GR_MXBOLD     | 150 | i64 e                      |
| GR_MHOLLOW    | 151 | i64 e                      |
| GR_MRAISED    | 152 | i64 e                      |
| GR_MSETTABG   | 153 | i64 t                      |
| GR_MRESTABG   | 154 | i64 t                      |
| GR_MFCOLORG   | 155 | i64 r, i64 g, i64 b        |
| GR_MBCOLORG   | 156 | i64 r, i64 g, i64 b        |

| Message        | Id  | Payload → reply                                                                                                                                              |
|----------------|-----|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| GR_MLOADPICT   | 157 | i64 p, str name → i64 0. The server fills its cache by the file transfer exchange if it does not hold the file; the reply comes after the picture is loaded. |
| GR_MPICTSIZE   | 158 | i64 p → i64                                                                                                                                                  |
| GR_MPICTSIZE   | 159 | i64 p → i64                                                                                                                                                  |
| GR_MPICTURE    | 160 | i64 p, x1, y1, x2, y2                                                                                                                                        |
| GR_MDELPIC     | 161 | i64 p                                                                                                                                                        |
| GR_MSCROLLG    | 162 | i64 x, i64 y                                                                                                                                                 |
| GR_MPATH       | 163 | i64 a                                                                                                                                                        |
| GR_MVIEWOFFG   | 164 | i64 x, i64 y                                                                                                                                                 |
| GR_MVIEWSCALE  | 165 | f64 x, f64 y                                                                                                                                                 |
| GR_MSCALEX     | 166 | i64 x → i64                                                                                                                                                  |
| GR_MSCALEY     | 167 | i64 y → i64                                                                                                                                                  |
| GR_MBLOCKCOPYG | 168 | i64 s, d, sx1, sy1, sx2, sy2, dx1, dy1, dx2, dy2                                                                                                             |
| GR_MOPENWIN    | 200 | open a window: i64 parent handle (0 for none), i64 new handle, i64 logical window id. The header wid is 0.                                                   |
| GR_MBUFFER     | 201 | i64 e                                                                                                                                                        |
| GR_MSIZBUFG    | 202 | i64 x, i64 y                                                                                                                                                 |
| GR_MGETSIZ     | 203 | → i64 x, i64 y                                                                                                                                               |
| GR_MGETSIZG    | 204 | → i64 x, i64 y                                                                                                                                               |
| GR_MSETSIZ     | 205 | i64 x, i64 y                                                                                                                                                 |
| GR_MSETSIZG    | 206 | i64 x, i64 y                                                                                                                                                 |
| GR_MSETPOS     | 207 | i64 x, i64 y                                                                                                                                                 |
| GR_MSETPOSG    | 208 | i64 x, i64 y                                                                                                                                                 |
| GR_MDRAGWIN    | 209 | (no payload)                                                                                                                                                 |
| GR_MSCNSIZ     | 210 | → i64 x, i64 y                                                                                                                                               |
| GR_MSCNSIZG    | 211 | → i64 x, i64 y                                                                                                                                               |
| GR_MSCNCEN     | 212 | → i64 x, i64 y                                                                                                                                               |
| GR_MSCNCENG    | 213 | → i64 x, i64 y                                                                                                                                               |
| GR_MWINCLIENT  | 214 | i64 cx, cy, ms → i64 wx, wy                                                                                                                                  |
| GR_MWINCLIENTG | 215 | i64 cx, cy, ms → i64 wx, wy                                                                                                                                  |
| GR_MFRONT      | 216 | (no payload)                                                                                                                                                 |
| GR_MBACK       | 217 | (no payload)                                                                                                                                                 |
| GR_MFRAME      | 218 | i64 e                                                                                                                                                        |
| GR_MSIZABLE    | 219 | i64 e                                                                                                                                                        |
| GR_MSYSBAR     | 220 | i64 e                                                                                                                                                        |
| GR_MMENU       | 221 | menu; an empty tree removes the menu                                                                                                                         |
| GR_MMENUENA    | 222 | i64 id, i64 e                                                                                                                                                |
| GR_MMENUSEL    | 223 | i64 id, i64 e                                                                                                                                                |
| GR_MSTDMENU    | 224 | i64 standard menu set, menu the user additions → menu, the combined menu as the server built it, which the client materializes for the program               |
| GR_MGETWINID   | 225 | → i64; wid 0, ids are global                                                                                                                                 |
| GR_MFOCUS      | 226 | (no payload)                                                                                                                                                 |
| GR_MGETWIGID   | 300 | → i64                                                                                                                                                        |

| Message             | Id  | Payload → reply                        |
|---------------------|-----|----------------------------------------|
| GR_MKILLWIDGET      | 301 | i64 id                                 |
| GR_MSELECTWIDGET    | 302 | i64 id, i64 e                          |
| GR_MENABLEWIDGET    | 303 | i64 id, i64 e                          |
| GR_MGETWIDGETTEXT   | 304 | i64 id → str                           |
| GR_MPUTWIDGETTEXT   | 305 | i64 id, str                            |
| GR_MSIZWIDGET       | 306 | i64 id, i64 x, i64 y                   |
| GR_MSIZWIDGETG      | 307 | i64 id, i64 x, i64 y                   |
| GR_MPOSWIDGET       | 308 | i64 id, i64 x, i64 y                   |
| GR_MPOSWIDGETG      | 309 | i64 id, i64 x, i64 y                   |
| GR_MBACKWIDGET      | 310 | i64 id                                 |
| GR_MFRONTWIDGET     | 311 | i64 id                                 |
| GR_MFOCUSWIDGET     | 312 | i64 id                                 |
| GR_MBUTTONSIZ       | 313 | str → i64 w, i64 h                     |
| GR_MBUTTONSIZG      | 314 | str → i64 w, i64 h                     |
| GR_MBUTTON          | 315 | i64 x1, y1, x2, y2, str, i64 id        |
| GR_MBUTTONG         | 316 | i64 x1, y1, x2, y2, str, i64 id        |
| GR_MCHECKBOXSIZ     | 317 | str → i64 w, i64 h                     |
| GR_MCHECKBOXSIZG    | 318 | str → i64 w, i64 h                     |
| GR_MCHECKBOX        | 319 | i64 x1, y1, x2, y2, str, i64 id        |
| GR_MCHECKBOXG       | 320 | i64 x1, y1, x2, y2, str, i64 id        |
| GR_MRADIOBUTTONSIZ  | 321 | str → i64 w, i64 h                     |
| GR_MRADIOBUTTONSIZG | 322 | str → i64 w, i64 h                     |
| GR_MRADIOBUTTON     | 323 | i64 x1, y1, x2, y2, str, i64 id        |
| GR_MRADIOBUTTONG    | 324 | i64 x1, y1, x2, y2, str, i64 id        |
| GR_MGROUPSIZG       | 325 | str, i64 cw, i64 ch → i64 w, h, ox, oy |
| GR_MGROUPSIZ        | 326 | str, i64 cw, i64 ch → i64 w, h, ox, oy |
| GR_MGROUP           | 327 | i64 x1, y1, x2, y2, str, i64 id        |
| GR_MGROUPG          | 328 | i64 x1, y1, x2, y2, str, i64 id        |
| GR_MBACKGROUND      | 329 | i64 x1, y1, x2, y2, i64 id             |
| GR_MBACKGROUNDG     | 330 | i64 x1, y1, x2, y2, i64 id             |
| GR_MSCROLLVERTSIZG  | 331 | → i64 w, i64 h                         |
| GR_MSCROLLVERTSIZ   | 332 | → i64 w, i64 h                         |
| GR_MSCROLLVERT      | 333 | i64 x1, y1, x2, y2, i64 id             |
| GR_MSCROLLVERTG     | 334 | i64 x1, y1, x2, y2, i64 id             |
| GR_MSCROLLHORIZSIZG | 335 | → i64 w, i64 h                         |
| GR_MSCROLLHORIZSIZ  | 336 | → i64 w, i64 h                         |
| GR_MSCROLLHORIZ     | 337 | i64 x1, y1, x2, y2, i64 id             |
| GR_MSCROLLHORIZG    | 338 | i64 x1, y1, x2, y2, i64 id             |
| GR_MSCROLLPOS       | 339 | i64 id, i64 r                          |
| GR_MSCROLLSIZ       | 340 | i64 id, i64 r                          |
| GR_MNUMSELBOXSIZG   | 341 | i64 l, i64 u → i64 w, i64 h            |
| GR_MNUMSELBOXSIZ    | 342 | i64 l, i64 u → i64 w, i64 h            |
| GR_MNUMSELBOX       | 343 | i64 x1, y1, x2, y2, l, u, id           |
| GR_MNUMSELBOXG      | 344 | i64 x1, y1, x2, y2, l, u, id           |
| GR_MEDITBOXSIZG     | 345 | str → i64 w, i64 h                     |
| GR_MEDITBOXSIZ      | 346 | str → i64 w, i64 h                     |
| GR_MEDITBOX         | 347 | i64 x1, y1, x2, y2, i64 id             |

| Message             | Id  | Payload → reply                                                         |
|---------------------|-----|-------------------------------------------------------------------------|
| GR_MEDITBOXG        | 348 | i64 x1, y1, x2, y2, i64 id                                              |
| GR_MPROGBARSIZG     | 349 | → i64 w, i64 h                                                          |
| GR_MPROGBARSIZ      | 350 | → i64 w, i64 h                                                          |
| GR_MPROGBAR         | 351 | i64 x1, y1, x2, y2, i64 id                                              |
| GR_MPROGBARG        | 352 | i64 x1, y1, x2, y2, i64 id                                              |
| GR_MPROGBARPOS      | 353 | i64 id, i64 pos                                                         |
| GR_MLISTBOXSIZG     | 354 | slst → i64 w, i64 h                                                     |
| GR_MLISTBOXSIZ      | 355 | slst → i64 w, i64 h                                                     |
| GR_MLISTBOX         | 356 | i64 x1, y1, x2, y2, slst, i64 id                                        |
| GR_MLISTBOXG        | 357 | i64 x1, y1, x2, y2, slst, i64 id                                        |
| GR_MDROPBOXSIZG     | 358 | slst → i64 cw, ch, ow, oh                                               |
| GR_MDROPBOXSIZ      | 359 | slst → i64 cw, ch, ow, oh                                               |
| GR_MDROPBOX         | 360 | i64 x1, y1, x2, y2, slst, i64 id                                        |
| GR_MDROPBOXG        | 361 | i64 x1, y1, x2, y2, slst, i64 id                                        |
| GR_MDROPEDITBOXSIZG | 362 | slst → i64 cw, ch, ow, oh                                               |
| GR_MDROPEDITBOXSIZ  | 363 | slst → i64 cw, ch, ow, oh                                               |
| GR_MDROPEDITBOX     | 364 | i64 x1, y1, x2, y2, slst, i64 id                                        |
| GR_MDROPEDITBOXG    | 365 | i64 x1, y1, x2, y2, slst, i64 id                                        |
| GR_MSLIDEHORISIZG   | 366 | → i64 w, i64 h                                                          |
| GR_MSLIDEHORISIZ    | 367 | → i64 w, i64 h                                                          |
| GR_MSLIDEHORIZ      | 368 | i64 x1, y1, x2, y2, i64 mark, i64 id                                    |
| GR_MSLIDEHORIZG     | 369 | i64 x1, y1, x2, y2, i64 mark, i64 id                                    |
| GR_MSLIDEVERTSIZG   | 370 | → i64 w, i64 h                                                          |
| GR_MSLIDEVERTSIZ    | 371 | → i64 w, i64 h                                                          |
| GR_MSLIDEVERT       | 372 | i64 x1, y1, x2, y2, i64 mark, i64 id                                    |
| GR_MSLIDEVERTG      | 373 | i64 x1, y1, x2, y2, i64 mark, i64 id                                    |
| GR_MTABBARSIZG      | 374 | i64 tor, i64 cw, i64 ch → i64 w, h, ox, oy                              |
| GR_MTABBARSIZ       | 375 | i64 tor, i64 cw, i64 ch → i64 w, h, ox, oy                              |
| GR_MTABBARCLIENTG   | 376 | i64 tor, i64 w, i64 h → i64 cw, ch, ox, oy                              |
| GR_MTABBARCLIENT    | 377 | i64 tor, i64 w, i64 h → i64 cw, ch, ox, oy                              |
| GR_MTABBAR          | 378 | i64 x1, y1, x2, y2, slst, i64 tor, i64 id                               |
| GR_MTABBARG         | 379 | i64 x1, y1, x2, y2, slst, i64 tor, i64 id                               |
| GR_MTABSEL          | 380 | i64 id, i64 tn                                                          |
| GR_MALERT           | 420 | str title, str message; wid 0                                           |
| GR_MQUERYCOLOR      | 421 | i64 r, g, b → i64 r, g, b; wid 0                                        |
| GR_MQUERYOPEN       | 422 | str → str; wid 0                                                        |
| GR_MQUERYSAVE       | 423 | str → str; wid 0                                                        |
| GR_MQUERYFIND       | 424 | str, i64 opt → str, i64 opt; wid 0                                      |
| GR_MQUERYFINDREP    | 425 | str, str, i64 opt → str, str, i64 opt; wid 0                            |
| GR_MQUERYFONT       | 426 | i64 fc, s, fr, fg, fb, br, bg, bb, effect → the same<br>nine, as chosen |
| GR_MEVENT           | 500 | an event, server → client on the event channel:<br>evt                  |